

**BABEȘ-BOLYAI UNIVERSITY, CLUJ-NAPOCA,
ROMANIA**

FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

Face2Learn

– MIRPR Report 2025 –

Team members:

Bucur Victor Sever, Popoviciu Luca, Porcar Cezar, Potra-Rațiu Darius, Preduca Matei

2025

Rezumat

Aplicația reprezintă un sistem inteligent bazat pe inteligență artificială, denumit **Face2Learn**, care combină modele de limbaj mari (LLM), sinteză vocală (TTS) și animație facială pentru a crea o experiență de învățare naturală și interactivă. Scopul proiectului este de a transforma informațiile academice în explicații conversaționale, vizuale și auditive, crescând astfel implicarea, accesibilitatea și retenția informației. Performanța sistemului este evaluată prin acuratețea răspunsurilor, latența end-to-end și naturalitatea percepută a avatarului.

Cuprins

1	Descrierea problemei rezolvate cu ajutorul AI	3
1.1	Contextul problemei	3
1.2	Scopul și importanța problemei	3
1.3	Utilizatorii sistemului	3
1.4	Datele de intrare și ieșire	4
1.5	Tipurile de date utilizate	4
1.6	Măsurarea performanței sistemului AI	4
2	Related work and useful tools and technologies	5
2.1	1. LoRA (Low-Rank Adaptation)	5
2.2	2. QLoRA	5
2.3	3. llama.cpp	5
2.4	4. TinyLLM (Phi, Mistral, TinyLLaMA)	6
2.5	5. RAGFlow	6
2.6	6. RAG-Anything	6
2.7	7. Whisper TTS	6
2.8	8. Piper	6
2.9	9. Wav2Lip	6
2.10	10. SadTalker	7
3	Evaluarea sistemului RAG utilizat în Face2Learn	8
3.1	Descrierea și explorarea datelor (EDA)	8
3.2	Descrierea algoritmului inteligent	9
3.3	Metodologia experimentală	9
3.4	Rezultate obținute	10
3.5	Analiza finală	10
4	Îmbunătățirea și Optimizarea Algoritmului Inteligent	11
4.1	Optimizarea Sistemului RAG prin Two-Stage Retrieval (Reranking)	11
4.1.1	Etapa 1: Candidate Generation (Bi-Encoder)	11
4.1.2	Etapa 2: Reranking (Cross-Encoder)	11
4.2	Arhitectura de Căutare Hibridă (Ensembling)	12

4.3	Optimizarea Modelului Generativ (QLoRA)	12
4.4	Augmentarea Datelor și a Interogărilor	12
4.4.1	Query Expansion (Multi-Query Generation)	12
4.5	Studiu Experimental al Hiperparametrilor	13
4.5.1	Schimbarea Modelului de Embedding	13
4.5.2	Analiza Temperaturii (T)	13
4.5.3	Analiza Top-K (Numărul de documente)	14
4.6	Optimizarea Alocării Resurselor și a Componentei Multimedia	14
4.6.1	Strategia Hibridă CPU-GPU	14
4.6.2	Optimizarea Generării Video (Wav2Lip)	14

Capitolul 1

Descrierea problemei rezolvate cu ajutorul AI

1.1 Contextul problemei

Sistemele educaționale tradiționale se bazează predominant pe text și prelegeri statice. În contextul actual al digitalizării, apare nevoia de metode de predare interactive, personalizate și accesibile. Proiectul **Face2Learn** propune utilizarea inteligenței artificiale multimodale pentru a crea un asistent virtual care explică concepte academice prin vorbire și expresii faciale sincronizate.

Modelul AI poate înțelege întrebări formulate în limbaj natural, poate accesa informații relevante din materiale educaționale și poate furniza răspunsuri clare, exprimate vocal și vizual printr-un avatar animat.

1.2 Scopul și importanța problemei

Scopul principal este de a transforma procesul de învățare într-o experiență naturală, captivantă și accesibilă. Importanța proiectului derivă din:

- creșterea **engagement-ului** și motivației elevilor/studentilor;
- asigurarea **accesibilității** pentru persoane cu deficiențe de vedere sau auz;
- sprijinirea cadrelor didactice prin automatizarea explicațiilor și a sesiunilor de Q&A;
- posibilitatea **învățării personalizate** în ritmul fiecărui utilizator.

1.3 Utilizatorii sistemului

- **Studenti și elevi** – folosesc avatarul AI pentru explicații și recapitulări;
- **Profesori și tutori** – utilizează sistemul pentru demonstrații și asistență automată;

- **Instituții educaționale** – integrează soluția pentru suport didactic 24/7;
- **Persoane cu dizabilități** – beneficiază de conținut multimodal adaptat.

1.4 Datele de intrare și ieșire

Date de intrare:

- întrebări formulate în limbaj natural (text sau voce);
- documente educaționale (manuale, cursuri, notițe);
- preferințe ale utilizatorului (limbă, voce, tonalitate).

Date de ieșire:

- răspuns text generat de LLM;
- voce sintetică naturală generată prin TTS;
- videoclip animat cu avatar sincronizat cu vorbirea.

1.5 Tipurile de date utilizate

- corpusuri textuale academice și explicații didactice;
- înregistrări audio pentru antrenarea TTS;
- imagini/video cu expresii faciale pentru sincronizare (lip-sync);
- embeddings semantice pentru căutare contextuală (RAG).

1.6 Măsurarea performanței sistemului AI

Performanța sistemului este evaluată prin indicatori cantitativi și calitativi:

- **Acuratețea răspunsurilor** – procentul de răspunsuri corecte;
- **Timpul mediu de răspuns** – durata procesării end-to-end;
- **Calitatea animației** – gradul de sincronizare buze-vorbire;
- **Consum de resurse** – memorie și timp de inferență.

Capitolul 2

Related work and useful tools and technologies

Această secțiune prezintă zece proiecte și tehnologii relevante pentru construcția unui avatar AI educațional. Pentru fiecare sunt menționate tipul datelor folosite, algoritmii utilizați, performanțele și tehnologiile implicate.

2.1 1. LoRA (Low-Rank Adaptation)

Date: text educațional.

Algoritmi: fine-tuning eficient al LLM-urilor prin adaptare low-rank.

Performanță: îmbunătățire a acurateței cu cost redus.

Tehnologii: PyTorch, Hugging Face, GitHub open-source.

2.2 2. QLoRA

Date: corpus text.

Algoritmi: fine-tuning cu cuantizare pentru reducerea memoriei.

Performanță: menține calitatea modelului la 4-bit.

Tehnologii: Transformers, bitsandbytes, Hugging Face.

2.3 3. llama.cpp

Date: text.

Algoritmi: inferență locală pentru modele cuantizate.

Performanță: latență redusă pe CPU.

Tehnologii: C++, GGUF models, GitHub.

2.4 4. TinyLLM (Phi, Mistral, TinyLLaMA)

Date: text.

Algoritmi: modele compacte pentru rulare eficientă.

Performanță: raport bun între viteză și acuratețe.

Tehnologii: PyTorch, Hugging Face.

2.5 5. RAGFlow

Date: documente și note de curs.

Algoritmi: RAG (retrieval augmented generation).

Performanță: răspunsuri mai relevante.

Tehnologii: LangChain, FAISS, GitHub.

2.6 6. RAG-Anything

Date: fișiere locale (PDF, text).

Algoritmi: flux simplificat RAG.

Performanță: acces rapid la surse externe.

Tehnologii: Python, Streamlit, GitHub.

2.7 7. Whisper TTS

Date: corpusuri audio și transcripții text.

Algoritmi: model neural de sinteză vocală bazat pe arhitectura Whisper.

Performanță: voce naturală și suport multilingv.

Tehnologii: PyTorch, Whisper TTS API (OpenAI), Hugging Face, GitHub.

2.8 8. Piper

Date: audio/text.

Algoritmi: TTS optimizat pentru dispozitive edge.

Performanță: latență foarte mică.

Tehnologii: Rust, on-device inference.

2.9 9. Wav2Lip

Date: video + audio.

Algoritmi: lip-sync bazat pe rețele CNN.

Performanță: aliniere buze-vorbire realistă.

Tehnologii: PyTorch, OpenCV, GitHub.

2.10 10. SadTalker

Date: imagine + audio.

Algoritmi: talking-face generation dintr-o singură imagine.

Performanță: expresii faciale naturale.

Tehnologii: PyTorch, DeepFace, GitHub.

Capitolul 3

Evaluarea sistemului RAG utilizat în Face2Learn

3.1 Descrierea și explorarea datelor (EDA)

Pentru evaluarea componentei de **întrebare-răspuns** din sistemul Face2Learn, a fost creat manual un set de date format din **50 de perechi întrebare-răspuns**. Fiecare întrebare a fost extrasă din conținutul manualului `manual2022.pdf`, iar răspunsul aferent reprezintă transcrierea exactă a fragmentului relevant.

Structura fișierului de date (`evaluare.json`):

```
[
  {
    "intrebare": "Ce reprezintă un segment orientat?",
    "raspuns_asteptat": "un segment ... unde s-a precizat originea si extremitatea"
  },
  ...
]
```

Preprocesare:

- Eliminarea diacriticelor pentru uniformitate.
- Conversia la litere mici și eliminarea spațiilor multiple.
- Nu s-a aplicat tokenizare, deoarece evaluarea folosește similaritate semantică și ROUGE.

Explorare sumară:

- Număr total de exemple: 100;
- Lungime medie întrebare: 9 cuvinte;

- Lungime medie răspuns: 12 cuvinte;
- Domeniu: concepte geometrice (vectori, segmente, egalitate etc.);
- Tip date: text scurt, conceptual – ideal pentru evaluarea unui sistem RAG.

3.2 Descrierea algoritmului inteligent

Algoritm ales: *Retrieval-Augmented Generation (RAG)*.

Motivație: Arhitectura RAG combină avantajele modelelor de căutare contextuală (retrieval) cu cele de generare (generation), permițând sistemului să răspundă coerent pe baza unui context relevant extras dintr-o bază de cunoștințe (PDF-ul cursului). Această abordare elimină necesitatea antrenării unui model mare de la zero, reducând costurile și riscul de halucinații.

Componente principale:

- **Retrieval:** Model de embedding `thenlper/gte-small` (din `sentence-transformers`) și indexare cu `faiss-cpu`. Scopul este transformarea fragmentelor PDF în vectori semantici și extragerea celor mai relevante pasaje.
- **Generation:** Modelul `mistral-7b-instruct-v0.3.Q4_K_M.gguf`, rulat local prin LM Studio. Acesta generează răspunsul final folosind contextul returnat de retriever.

Librării utilizate: `langchain`, `sentence-transformers`, `faiss-cpu`, `numpy`, `transformers`, `sklearn.metrics`.

3.3 Metodologia experimentală

Împărțirea datelor: Setul de 50 de exemple a fost folosit în întregime ca **set de test**. Scopul principal a fost evaluarea sistemului complet (RAG + LLM), nu antrenarea unui model nou.

Hiperparametri utilizați:

Parametru	Valoare	Descriere
<code>chunk_size</code>	256	Lungimea unui fragment de text la indexare
<code>chunk_overlap</code>	25	Suprapunerea dintre fragmente consecutive
<code>k</code>	4	Numărul de pasaje returnate de retriever
<code>temperature</code>	0.3	Controlul creativității LLM-ului

Metrici de evaluare:

- **ROUGE-L (F1):** măsoară suprapunerea exactă între răspunsul generat și cel așteptat;
- **Similaritate Semantică (Cosine Similarity):** măsoară apropierea conceptuală dintre răspunsuri, folosind embedding-urile `gte-small`.

3.4 Rezultate obținute

Metrică	Valoare medie
ROUGE-L (F1)	20.61%
Similaritate Semantică	90.24%

Interpretare:

- Scorul semantic ridicat (90%) indică faptul că sistemul a înțeles corect întrebările și a generat răspunsuri cu același sens;
- Scorul ROUGE redus (20%) arată că modelul preferă să reformuleze textul în loc să reproducă exact pasajul original.

Exemple reprezentative:

Caz	Întrebare	Răspuns așteptat	Răspuns generat	Observație
1	Segment orientat	un segment ... cu origine și extremitate	o porțiune ... cu direcție asignată	sens corect, formulare diferită
2	Egalitate segmente	A=C și B=D	A=D și B=C	halucinație (ordine inversată)
3	Vector liber	o clasă de echivalență	o familie de vectori legați	parafrizare semantică

3.5 Analiza finală

Diferența semnificativă dintre scorurile ROUGE și Similaritate Semantică evidențiază o problemă frecventă în sistemele RAG:

- Modelul LLM **înțelege contextul**, dar nu respectă instrucțiunea „răspunde exclusiv pe baza textului oferit”;
- Răspunsurile sunt logic corecte, dar lexical diferite;
- În unele cazuri apar **halucinații minore** (exemplul #2).

Capitolul 4

Îmbunătățirea și Optimizarea Algoritmului Inteligent

În această etapă a proiectului, ne-am concentrat pe îmbunătățirea algoritmului din punct de vedere al calității procesului de rezolvare (acuratețe semantică) și al complexității spațiale (portabilitate), implementând tehnici avansate de Machine Learning și optimizând resursele hardware.

4.1 Optimizarea Sistemului RAG prin Two-Stage Retrieval (Reranking)

Pentru a reduce fenomenul de *halucinație* și a crește relevanța contextului, am implementat o arhitectură de căutare în două etape (*Two-Stage Retrieval*).

4.1.1 Etapa 1: Candidate Generation (Bi-Encoder)

- **Tehnică:** Căutare Vectorială (*Semantic Search*).
- **Model:** `thenlper/gte-small` (Bi-Encoder).
- **Proces:** Interogarea utilizatorului și fragmentele din documente sunt transformate în vectori (embeddings). Se folosește indexarea FAISS pentru a extrage rapid un număr extins de candidați ($k = 20$).
- **Scop:** Maximizarea **Recall-ului** (aducerea informației relevante, chiar dacă include zgomot).

4.1.2 Etapa 2: Reranking (Cross-Encoder)

- **Tehnică:** Reordonare bazată pe relevanță contextuală.
- **Model:** `cross-encoder/ms-marco-MiniLM-L-6-v2`.

- **Proces:** Cei 20 de candidați sunt analizați de un model Cross-Encoder împreună cu întrebarea, calculând un scor de relevanță (*logit*) pentru fiecare pereche.
- **Rezultat:** Se selectează top 5 documente cu scorul cel mai mare pentru a fi trimise la LLM, maximizând astfel **Precizia**.

4.2 Arhitectura de Căutare Hibridă (Ensembling)

Am implementat o strategie de **Hybrid Search** care combină:

1. **Căutare Semantică (Dense Retrieval):** Utilizează embedding-uri și FAISS pentru a înțelege contextul și sinonimele.
2. **Căutare Lexicală (Sparse Retrieval):** Utilizează algoritmul **BM25** pentru a găsi termeni exacti (ex: „Teorema lui Ceva”).

Fuziunea rezultatelor se realizează prin algoritmul **Reciprocal Rank Fusion (RRF)**, asigurând un echilibru între înțelegerea semantică și precizia terminologică.

4.3 Optimizarea Modelului Generativ (QLoRA)

Pentru a adapta stilul de comunicare și a optimiza complexitatea spațială, am utilizat tehnica **QLoRA** (Quantized Low-Rank Adaptation).

- **LoRA Adapters:** Am înghețat modelul de bază și am antrenat doar matrice de rang mic injectate în straturile de atenție ($< 1\%$ parametri antrenabili).
- **Quantization (NF4):** Modelul a fost încărcat în precizie de 4-biți, reducând necesarul de memorie VRAM de la $\approx 16\text{GB}$ la $\approx 6\text{GB}$, permițând rularea pe un singur GPU consumer (RTX 4060).
- **Regularizare:** S-au aplicat **Weight Decay** și **LoRA Dropout** pentru a preveni overfitting-ul.

4.4 Augmentarea Datelor și a Interogărilor

4.4.1 Query Expansion (Multi-Query Generation)

Am implementat un mecanism de expandare a interogării pentru a rezolva ambiguitatea. Un apel LLM intern generează 3 variante semantice ale întrebării utilizatorului, sistemul execută căutări paralele pentru toate variantele și agregă rezultatele, îmbunătățind semnificativ acoperirea (Recall).

4.5 Studiu Experimental al Hiperparametrilor

Am efectuat o analiză detaliată a impactului hiperparametrilor asupra performanței sistemului.

4.5.1 Schimbarea Modelului de Embedding

Trecerea de la `gte-small` la `paraphrase-multilingual-MiniLM-L12-v2` a adus îmbunătățiri majore în procesarea limbii române.

Caracteristică	Model Inițial (gte-small)	Model Final (MiniLM)
Limba	Preponderent Engleză	Multilingv (50+ limbi)
Înțelegere	Cuvinte cheie	Sens semantic profund
Performanță RAG	Returna uneori pasaje irelevante	Precizie ridicată în regăsire

4.5.2 Analiza Temperaturii (T)

Am testat impactul temperaturii asupra metricilor de evaluare (ROUGE-L, BLEU, Semantic Similarity).

- **Temperatura normală (0.1):**

- Similaritate Semantică: 91.54%
- ROUGE-L: 19.23%
- BLEU: 3.92%
- *Observație:* Echilibrul ideal între precizie și naturalitate.

- **Temperatura 0 (Determinist):**

- Similaritate Semantică: 97.23%
- ROUGE-L: 32.46%
- BLEU: 8.34%
- *Observație:* Scoruri maxime, dar răspunsul este rigid, pierzând „touch-ul” explicativ și pedagogic dorit.

- **Temperatura 0.2 (Creativ):**

- Similaritate Semantică: 82.23%
- ROUGE-L: 9.98%
- BLEU: 2.33%

- *Observație:* Scădere drastică a performanței; modelul începe să halucineze și să devieze de la textul sursă.

Concluzie: Am selectat valoarea $T = 0.1$ pentru a menține rigoarea academică fără a sacrifica claritatea explicației.

4.5.3 Analiza Top-K (Numărul de documente)

- **Normal** ($k = 4$): Similaritate 91.54%, ROUGE 19.23%.
- $k = 2$: Scădere ușoară a scorurilor (Similaritate 91.18%, ROUGE 18.30%). Risc de a pierde context vital.
- $k = 6$: Creștere neglijabilă (Similaritate 91.35%, ROUGE 19.74%), dar cu o latență crescută nejustificat.

Decizie: S-a păstrat valoarea $k = 4$ ca optim între performanță și viteză.

4.6 Optimizarea Alocării Resurselor și a Componentei Multimedia

4.6.1 Strategia Hibridă CPU-GPU

Pentru a gestiona eficient memoria limitată (8GB VRAM), am adoptat o arhitectură distribuită:

- **CPU:** Gestionează baza de date vectorială (FAISS) și modelele de embedding, care au cerințe mici de calcul.
- **GPU:** Dedicat exclusiv modelului generativ (Llama 3) și, secvențial, procesului de generare video.

4.6.2 Optimizarea Generării Video (Wav2Lip)

Am înlocuit modelul inițial **SadTalker** cu **Wav2Lip** pentru a optimiza timpul de răspuns și calitatea sincronizării.

- **Viteză:** Wav2Lip reduce timpul de generare de la un raport de 5:1 (SadTalker) la aproape 1:1 (real-time).
- **Calitate:** Oferă o sincronizare labială („lip-sync”) superioară, esențială pentru credibilitatea unui avatar educațional, eliminând artefactele vizuale prezente la modelele anterioare.